

University of Economics, Prague

Faculty of Informatics and Statistics



MODEL DRIVEN DEVELOPMENT AND ITS USE IN DEVELOPMENT OF LARGE INFORMATION SYSTEMS

Bachelor thesis

Study programme: Applied informatics

Field of study: Applied informatics

Author: Adéla Barchánková

Supervisor: Ing. Iva Stanovská

Prague, May 2020

Declaration

I hereby declare that I am the sole author of the thesis entitled “Model driven development and its use in development of large information systems“. I duly marked out all quotations. The used literature and sources are stated in the attached list of references.

In Prague on 10.05.2020

.....

Adéla Barchánková

Acknowledgement

I hereby wish to express my appreciation and gratitude to the supervisor of my thesis: Ing. Iva Stanovská, for her time, dedication and many valuable advice.

Abstract

The main aim of this bachelor thesis is to provide a comprehensive description of Model-driven development and analyse its approaches, practices and available tools with focus on server-based applications of large information systems.

The first part of this study shall cover the definitions of the methodology, its principles, best practices, and analyse the possible benefits or disadvantages of this approach. It will also capture the current situation on the market with commercial MDD tools.

Second part contains an analysis and description of the Metasystem application: a non-commercial MDD tool developed by Czech IT company, which is used for modelling and development of large information systems. The thesis provides a comparison between the mentioned tool and one of the most commonly used application – Sparks's Enterprise Architect with one main question: Would Metasystem be commercially competitive on current market? In the latter part, the thesis focuses on an analysis and design of possible enhancements in the functionality of the tool.

Keywords

software development, model-driven development, modelling, information technology

Abstrakt

Hlavním cílem této bakalářské práce je podat srozumitelný popis přístupů, praktik a dostupných nástrojů k vývoji řízenému modely (častěji anglicky “model-driven development”) se zaměřením na serverové aplikace robustních informačních systémů.

První část práce se soustředí na definice metodologie, její principy, nejlepší praktiky a analyzuje možné výhody či nevýhody tohoto přístupu. Zároveň zachycuje aktuální situaci na trhu s nástroji pro MDD.

Druhá část obsahuje analýzu a popis aplikace Metasystem: nekomerčního nástroje vyvíjeného v dále zmíněné české IT společnosti, který se využívá k modelování a vývoji robustních informačních systémů. Práce porovnává zmíněný nástroj s jedním z nejčastěji využívaných komerčních nástrojů pro MDD – aplikací Enterprise Architect s cílem zodpovědět, zda by byl Metasystem konkurence schopný na trhu. Poslední část práce je zaměřena na návrh a analýzu rozšiřující funkcionality do této aplikace.

Klíčová slova

vývoj software, vývoj řízený modely, modelování, informační technologie

Content

List of figures.....	9
Introduction	10
Literature review	11
Model-driven development	11
The Pragmatics of Model-Driven Development.....	11
Patterns: Model-Driven Development Using IBM Rational Software Architect	12
Model-driven development	13
1.1 What is “a model”	13
1.1.1 Ways to depict a model	14
1.2 MDD principles.....	15
1.2.1 Models must be precise and readable for machines	15
1.2.2 Transformations are automated	16
1.2.3 Better business-IT alignment	17
1.3 Advantages and disadvantages	18
1.4 Standards and initiatives	18
1.4.1 Object Management Group	19
2 Commercial MDD tools	21
2.1 Enterprise Architect	21
2.1.1 Pricing and editions	21
2.1.2 Functionalities	22
2.2 Power Designer	23
2.2.1 Pricing	23
2.2.2 Functionalities.....	23
2.3 Rational Software Architect	24
2.3.1 Pricing	24
2.3.2 Functionalities.....	24
2.4 Conclusion	24
3 Metasystem application.....	26
3.1 Origins and motives	26
3.2 Framework and functionalities	27
3.3 Comparison with commercial tools	28
3.3.1 Metasystem	28
3.3.2 Enterprise Architect	33

3.3.3 Comparison and summary	37
4 Analysis and design of new MTS functionality	39
4.1 Class inheritance and polymorphism.....	40
4.1.1 Initial study	40
4.1.2 Requirements specification	41
4.1.3 Analysis and design	43
4.2 Message modelling enhancement.....	46
4.2.1 Initial study	48
4.2.2 Requirements specification	49
4.2.3 Analysis and design.....	51
Conclusions	55
Terminological glossary.....	56
List of references	58

List of abbreviations

ASW	Application Software
BT	Bachelor's Thesis
BPMN	Business process model and notation
EA	Enterprise Architect
FIS	Faculty of Informatics and Statistics
HTML	Hypertext mark-up language
IS	Information system
IT	Information technology
MDE	Model-driven engineering
MTS	Metasystem
OOP	Object-oriented programming
PIM	Platform-independent model
PDM	Platform-dependent model
SW	Software
TOGAF	The Open Group Architecture Framework
UML	Unified modelling language
WSDL	Web service definition language
XML	Extensible mark-up language
XMI	XML Metadata interchange
XSD	XML Definition language

List of figures

Figure 1: language's abstraction level and the degree of abstraction [2]	15
Figure 2: Enterprise Architect v15.1 workspace	22
Figure 3: Power designer v16.7 workspace	23
Figure 4: Class diagram in Metasystem application.....	29
Figure 5: Metasystem class method definition	30
Figure 6: Metasystem message definition	31
Figure 7: Metasystem state machine	32
Figure 8: Metasystem transition definition	32
Figure 9: Enterprise Architect Conceptual model	34
Figure 10: Enterprise Architect Logical model	34
Figure 11: Enterprise Architect Sequence diagram.....	35
Figure 12: Enterprise Architect Communication diagram	36
Figure 13: Metasystem "Add new derived class" dialogue	44
Figure 14: Metasystem "Select the parent class".....	44
Figure 15: Metasystem Class artefact detail	45
Figure 16: Metasystem Inherited method detail.....	45
Figure 17: Metasystem Add overloaded method.....	46
Figure 18: Metasystem Select method for overloading	46
Figure 19: Metasystem overloaded methods selection	46
Figure 20: Metasystem Message builders	47
Figure 21: Metasystem cloning builders.....	48
Figure 22: Metasystem "Remove all rules for sub-elements"	51
Figure 23: Metasystem "Remove all conditions for sub-elements"	52
Figure 24: Metasystem "Remove artefacts for mapping for all sub-elements"	52
Figure 25: Metasystem "Remove artefacts for mapping to message for all sub-elements"	52
Figure 26: Metasystem "Remove all rules for selected elements"	53
Figure 27: Metasystem "Select element for action"	53
Figure 28: Metasystem multiplication of builders	54

Introduction

In the current world, where information technology is still one of the most rapidly growing industries, it might be hard to find one's way around in all existing methodologies, approaches and best practices. The era of first electronic computers is said to have begun already in 1940s and from that, the field has come a long way. First attempts to formalize the used practices emerged in 1960s, connected with the introduction of first high-level programming languages, such as Fortran, COBOL, or Lisp.

From then until now, a big amount of methodologies, frameworks and approaches for software development processes were introduced. When it comes to the real world, they are in fact never used in their absolute and natural form, but they are combined and modified, which can in the end just add up to the possible confusion.

In this thesis, I focus on one of the mentioned phenomena: Model-driven development. It appears in many forms, such as Model-driven engineering, Model-driven architecture and more. They share common principles and approaches, but as their definitions suggest, they focus on slightly different parts of the information technology field. Nonetheless as I aim my work to the software development process itself, I stayed with the Model-driven development.

My goal is to provide a comprehensive overview of model-driven development principles, advantages, disadvantages and to compare currently used MDD tools with a non-commercial application Metasystem (further also MTS). The comparison is done mainly experimentally, where I attempt to model a simple application with the compared tools, or by analysis of secondary data. As a result of this investigation, I evaluated the possible weak points of the MTS application and designed two new features, which could enhance the development possibilities in it.

Main premise for the practical research is certain amount of abstraction from infrastructural matters, the exploration is focused mainly on the modelling itself. Other limitations originate in the fact, that the commercial tools are usually not usable for complex application development without some degree of customization.

As a result, I expect my work to provide an elaborate analysis on the matter, whether the currently used Metasystem application is beneficial, whether it would be commercially successful, and to offer a possible way how to enhance it in the future.

Literature review

This chapter aims to summarize the main literature used as knowledge base for this thesis. Mentioned publications inspired me, helped me organize my thoughts and provided me with the expertise needed to finish this work.

Model-driven development

Stephen J. Mellor, Anthony N. Clark, Takao Futugami

Published in IEEE Software 5/2003 [2]

This publication very grippingly describes the main principles and premises for model-driven development. The authors focus the bigger part of the work to a definition of a model itself, its usage and its possibilities with a main target to enlighten the MDD advantages in a simple and intriguing way, making it understandable also for people without any deep technical education or experience. The thought I found the most interesting was that almost everything in the information technology field, such as application code written in Java or C# code, can be considered a model – simply because it describes some matter on certain level of abstraction and in the end, it is automatically transformed into the resulting product.

The Pragmatics of Model-Driven Development

Bran Selic

Published in IEEE Software 5/2003 [1]

The Pragmatics of Model-Driven development is again quite interesting publication by Canadian engineer and OMG cochair Bran Selic. In the beginning, Selic pays a lot of attention to the attitude of current society towards model-driven techniques. He also emphasises the MDD potential and calls it “first true generational shift in basic programming technology since the introduction of compilers”. Then the work focuses on detailed description of five key characteristics, which models must possess. The last part then discusses the model executability, scalability and efficiency of automatically generated code. All thoughts that Selic expresses in his work are definitely valid and easily understood, however it is recognizable, that it is almost 17 years old and since then, information technologies have progressed significantly.

Patterns: Model-Driven Development Using IBM Rational Software Architect

IBM Redbooks [5]

This book combines both the theoretical and practical knowledge. It consists of two parts, each then contains several chapters. For my thesis, I benefited mainly from the first part called “Approach”. In six chapters, the authors introduce the model-driven development concepts, approaches, solutions lifecycle and project planning. The second part “Implementation” was more practical and focused on work with IBM Rational Software Architect. That helped me mainly for the part of my thesis, where I describe the commercial tools for MDD and aim to provide a comparison with the Metasystem application.

Model-driven development

This chapter aims to describe and explore the model-driven development principles and objectives. They are important for understanding the matter itself and they serve as the basis, above which the targeted analysis and comparison is carried out in the following chapters.

Model-driven development is a software development methodology. The main focus lies on the construction of models, which are then through various ways transformed into the resulting application.

These days, software solutions are becoming more and more complex, both because of a great range of used technologies which continuously evolve and because of the business and IT alignment, which thanks to worldwide computerisation also grows exponentially. This creates new challenges in related fields, such as project management or human resources. Model-driven development aspires to tackle these by increasing the level of abstraction during application development, which creates space for a better insight into matters, which were once understandable solely to skilled professionals.

The key promise is that the final code of software programs is generated automatically.

1.1 What is “a model”

Model is depicted as a reflection of reality, generally said. Models are used throughout many fields of human knowledge – from art, through science, all the way to engineering. This thesis will focus mainly on the last-mentioned: software engineering.

“A model is a coherent set of formal elements describing something (for example, a system, bank, phone or train) built for some purpose, that is amenable to a particular form of analysis.” [2]

The main principle of models, in general, is an abstraction. The level of it may differ, depending on the model’s purpose and target. For a basic introduction to some subject matter, a brief and simple model will be sufficient to provide the necessary overview to a mass of people. But in the same way, a detailed and complex model of this matter will

easily transfer needed knowledge to a group of specialized experts. To sum it up, the higher the level of abstraction is, the closer it is to the common human understanding.

Another key characteristic is understandability: models should be a way to reflect any complex matter in a way, which is easily understandable with common sense and without any deep expert knowledge – of course only when modelled with the correct amount of abstraction, as mentioned previously. The goal of any model should be to reduce the needed level of used intellect to understand the modelled matter.

1.1.1 Ways to depict a model

Each model is expressed in a certain language. In this context, “language” does not necessarily signify a mean of communication between humans, but a way how to transfer any kind of knowledge between various entities (humans, computers, animals, etc.).

Languages itself also have some abstraction level, which is defined by its syntax and semantics. If we abstract from common human languages and dive more into the information technologies field, we can find a very nice example of language abstraction when comparing following three: UML (Unified Modelling Language), C# and Assembly language. All can describe the same matter (e.g. Classes of some simple application), but each in its own level of detail.

Using UML, we can easily visualize the main classes, their attributes, possibly the names of their methods and the relations between the classes. The same subject matter described the C# programming language would allow us to understand also the implementation of the methods and maybe also the way of persistence of the objects. And if we take it another step lower, to the for-example Assembly language, we would be suddenly able to see and understand the allocation of the computer memory and processor usage. And if we took the matter even further, even Assembly language abstracts on some level – over the machine code.

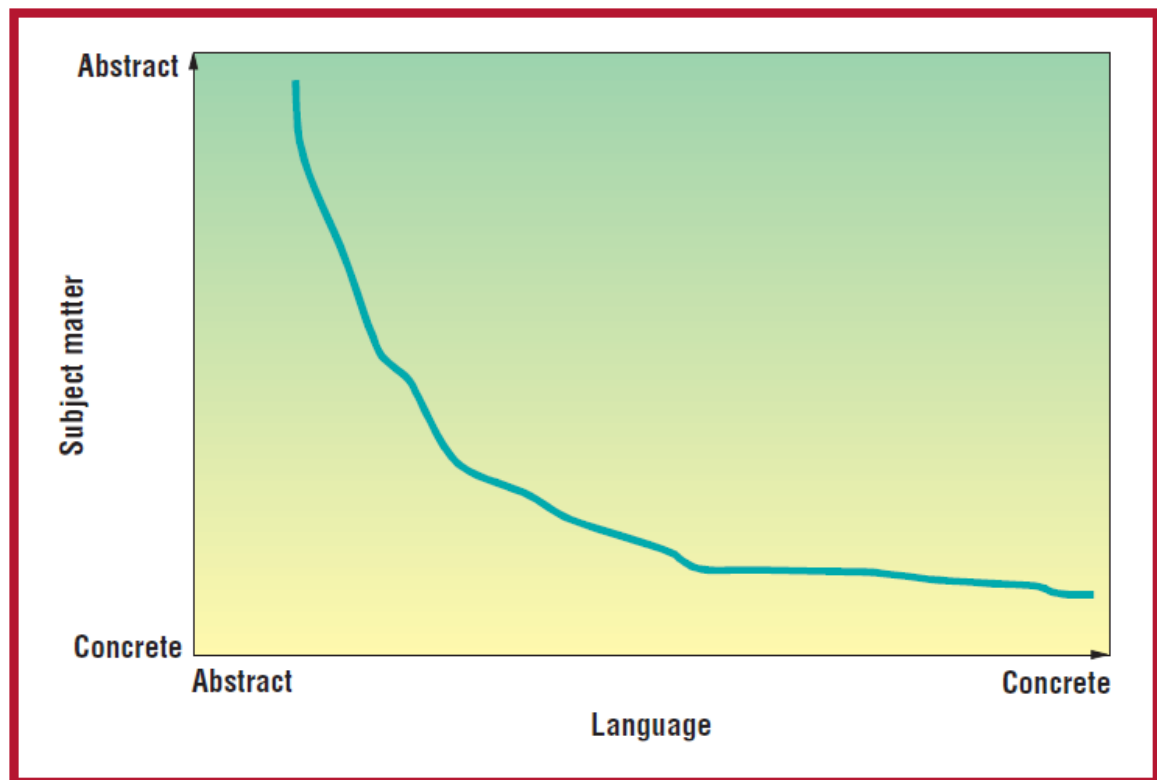


Figure 1: language's abstraction level and the degree of abstraction [2]

If we would apply this logic entirely, even standard programming could be considered as modelling. And according to some definitions, it really is. Nevertheless, it is not commonly accepted and it is not a principle of Model-driven development.

The way of how the model is depicted is very important for one of the aspects of MDD: automatization. As the main goal of Model-driven development is an automated transformation of models from source form to target form, the ability to describe and outline the used modelling language or technique is a necessity.

To formalize the used modelling method, it is common to create a model of the modelling language – the so-called metamodel. A standard for definition and creation of metamodels is the OMG Meta-Object Facility QVT, more described in 1.3 Standards and initiatives.

1.2 MDD principles

1.2.1 Models must be precise and readable for machines

In order to use model-driven development practices, all created models must be machine readable. That means mainly following: they must be in the computerised form (not on a paper) and they must be describable by some modelling standards. Only then they can be

automatedly transformed into various artefacts on different levels of abstractions, such as platform-dependent models and the code itself.

Another point which supports this is that models must be relatively precise. Relatively, because the used transformations differ and often they are tailor-made, therefore it is hard to define the needed level of precision for the models in general. But if the initial representation of the matter is not standardized, we can hardly expect the automated transformations to provide worthy result.

This leads to the fact, that even though modelling can be considered a creative activity with high level of freedom, it still needs to have strictly defined boundaries and techniques. However, if they are determined well, the modeller should still have enough space for expressing own inventiveness.

Mentioned principle is supported by many standards and initiatives, such as UML (Unified Modelling Language). More on this in 1.4.

1.2.2 Transformations are automated

As it was already mentioned, automated transformation is the key promise for Model-driven development. It allows to encapsulate the best practices, skill and knowledge of the developers and what is most important, maintain and apply it.

Transformations can be of various kinds. Some are default and often not even considered – such as the transition from application code to the native binary computer language. On the other side, many of them are tailor-made and designed for specific transformations of different artefacts.

The results of such transformations can vary as well, depending on the transformation type and pattern. When talking about Model-driven development, most of people imagine as a result an application code – Java, C# or any other. However, this is not entirely true, often the outcome can be:

- another model (e.g. transformation between platform-independent model to platform-specific),
- documentation,
- test artefacts (for example automated testing scenarios, unit tests etc.),
- installation scripts,

and certainly, many more.

The automated transformations in ideal scenarios lead to:

- improved productivity (many tasks are performed automatically),
- better knowledge reuse (the knowledge is encapsulated in the transformation pattern),
- consistency and lower error rate.

1.2.3 Better business-IT alignment

As models in general are much closer to human-to-human communication and easier understandable than application code, Model-driven development should enable much better alignment between the target business and IT.

This allows mainly to simplify communication between the stakeholders and the developers. In standard programming methodologies, understanding the business needs and transferring the knowledge to the programmers and development-responsible people can be very challenging and time-consuming activity. Moreover, mainly for tailor-made software solutions where the business can be very complicated, this can result in many misunderstandings and errors in the final application.

On the other hand, development based on model-driven techniques offers both easier understanding of the matter and unambiguous interpretation between all parties. While a textual description can be often comprehended in various ways, if a model is depicted in standardized form, it is in most cases clear and definite.

Mentioned features have one aspect that is more important – even people, who do not have deep programming knowledge, can create the models. If the modelling process and following transformation have well defined boundaries and techniques, they can be done by analysts and in general people closer to the business itself. This then may result in better and more accurate business behaviour of the application.

Moreover, in ideal cases, the mentioned points might also simplify another key matter in the software development field – the human-resources. Model-driven principles should ease the knowledge transfer and therefore enable the companies to be more flexible on the employee market. Nonetheless, even this fact has two sides to it - if the used model-driven framework is very specific or tailor-made, it can also lead to the situation, where the key people who maintain it have such particular know-how, that they become irreplaceable and their loss may result in complete failure of the company.

1.3 Advantages and disadvantages

“Engineering models aim to reduce risk by helping us better understand both a complex problem and its potential solutions before undertaking the expense and effort of a full implementation.” [1]

There are several discussions, which aim to explore, whether modelling and model-driven development in general is beneficial.

Between main arguments against belongs:

- models which are used for documentation purposes take a lot of work to create, but usually cannot flexibly reflect changes
- analysts, who often create the models, have a different attitude to the matter than developers: they often do not bear in mind the implementation requirements and conditions
- difficult backwards traceability: once the model is transformed into the resulting application, it is hard to transform it back to the model and clearly connect the specific part of the application with a specific part of the model (which is crucial e.g. for debugging or amendment of small functionalities)

But most of the arguments are easily diverted with just one word: automation. Experts have agreed that once the transformation and execution of the models is fully automated, majority of the mentioned issues are much less applicable.

As Stephen Mellor, Anthony Clark and Takao Futagami mentioned in Model-Driven Development article published in IEEE Software:

“Automation does not remove the requirement for creativity. Rather, it formalizes existing solutions and raises the level at which we can apply creativity, thus giving the developer more leverage. In turn, we must now ask how we can produce developers that think at a level of abstraction above the currently fashionable programming language or technology.”[2]

1.4 Standards and initiatives

This chapter includes a brief overview of the commonly known and recognized MDD standards and initiatives. Probably the most widespread are the ones presented by Object Management Group (OMG).

1.4.1 Object Management Group

Object Management Group is an international standards consortium. It was founded in year 1989 by eleven companies (such as HP, IBM, Sun, Apple etc.) and until now, it has 11 ratified ISO standards.

Unified Modelling Language

UML is a standard language used for visualizing and modelling of software systems. Its development began between 1994-1995 at Rational Software, in 1997 it was overtaken as a standard by OMG.

It allows to visualize systems at various abstraction levels, either from static or dynamic point of view. UML has various types of diagrams, including following:

- activity diagram
- state machine diagram
- use case diagram
- sequence diagram
- class diagram
- component diagram

and many others.

Meta-Object Facility

Meta-Object Facility, or MOF, is an ISO standard for model-driven development. It formalizes methods and principles how to define, create and understand metamodels. In addition, it specifies techniques for handling of models – their versioning, life cycles, transformations and more.

XML Metadata Interchange

XMI is an Object Management Group standard for exchange of metadata using Extensible Mark-up Language (XML). It is most commonly used for interchange of UML models, though it can be used for any metadata which can be expressed in Meta-Object Facility.

Common Object Request Broker Architecture

CORBA is another standard introduced by the OMG. It is meant to allow collaboration of systems on different platforms – both software (operating systems, programming

languages) and hardware. It uses so called interface description language, in which it specifies the interface and mapping to concrete implementation.

Main components are so called object request brokers (ORB) – a middleware allowing program calls between computers via network.

2 Commercial MDD tools

Following chapter focuses on brief description and analysis of selected current commercial market with tools, which support Model-Driven engineering.

I chose following applications mainly based on the general popularity, but also because they aim for the same target and results as the custom modelling tool described in the next part of the theses and therefore would be possible substitute candidates for it.

For better comparison of the mentioned products, I have also described the pricing conditions. As they all have very similar functionality range, the pricing can be for many customers the most important aspect.

2.1 Enterprise Architect

Developer: Sparx Systems

Enterprise Architect is a collaborative tool for analysis, design, modelling and maintaining systems. It offers support for full lifecycle of software development – from architecture design and requirements management up to testing.

2.1.1 Pricing and editions

EA is offered in 4 different editions:

- Professional (Entry level UML modelling for individuals and workgroups)
- Corporate (Robust team-based modelling and design platform)
- Unified (Advanced modelling, design, simulation and frameworks)
- Ultimate (Fully featured)

For each edition, Sparx offers two variants of licences: standard and floating. Floating offers the license to be used by various users – a licence on the product is only required, while the user has the software running. Once it is shut down, the license is available for another user to use it.

Both types are one-time purchase. The prices for a license differ from \$229 (Professional edition, standard license) to \$899 (Ultimate edition, floating license). Application is also offered as free 30 days trial.

2.1.2

2.1.2 Functionalities

Enterprise Architect offers a wide range of techniques and functionalities for complete system analysis, design and implementation support.

Its modelling tools are based on open standards as UML, BPMN or SysML. There is also build-in support for many enterprise architecture frameworks like TOGAF or UPDM.

EA contains also features for team collaboration and project management – task tracking system, burn down charts and other useful diagrams.

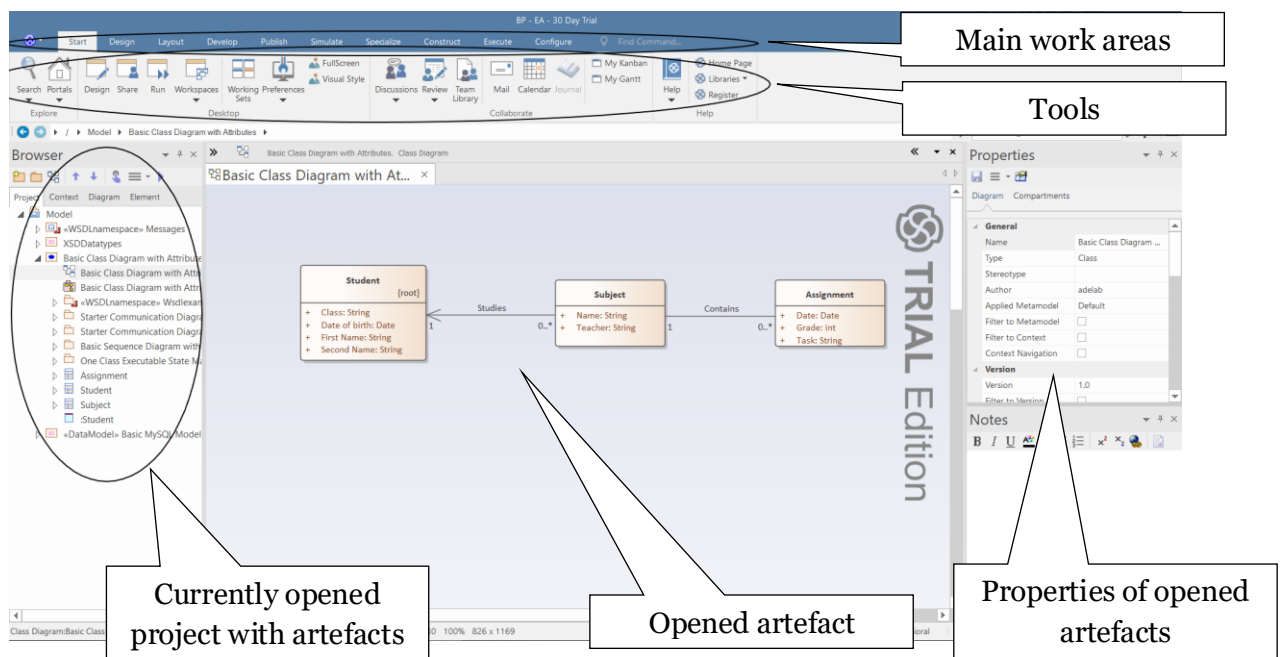


Figure 2: Enterprise Architect v15.1 workspace

Each of the main work areas offers different functions for the user – one is focused on development, one on project management, one on generation, etc. Its tools are then offered in the section below. The rest of the screen is then used for viewing the current project and selected artefacts.

As the best quality, reviewers often consider mainly the high flexibility, wide range of functionalities and extensive support of many platforms and standards. However, it is also often mentioned that as it has so many features, it is often confusing and might be difficult to use without proper training.

2.2 Power Designer

Developer: SAP (originally Sybase Inc.)

Power Designer is collaborative modelling application, which supports business process modelling, data modelling, code generation and much more. It can run either as a native app on Windows or through a plugin in Eclipse environment.

2.2.1 Pricing

It is not possible to find relevant pricing information about Power designer, nevertheless website www.powerdesigner.biz powered by Novalis, there are defined two possible editions: Power Designer Data Architect which is suited mainly for data modelling and database design, and Power Designer Enterprise Architect, which contains all functionalities provided.

The stated price for a license is \$4,299, but also with a side note, that the price might differ based on country and needed licenses.

2.2.2 Functionalities

Power designer is mainly a modelling and transformation tool.

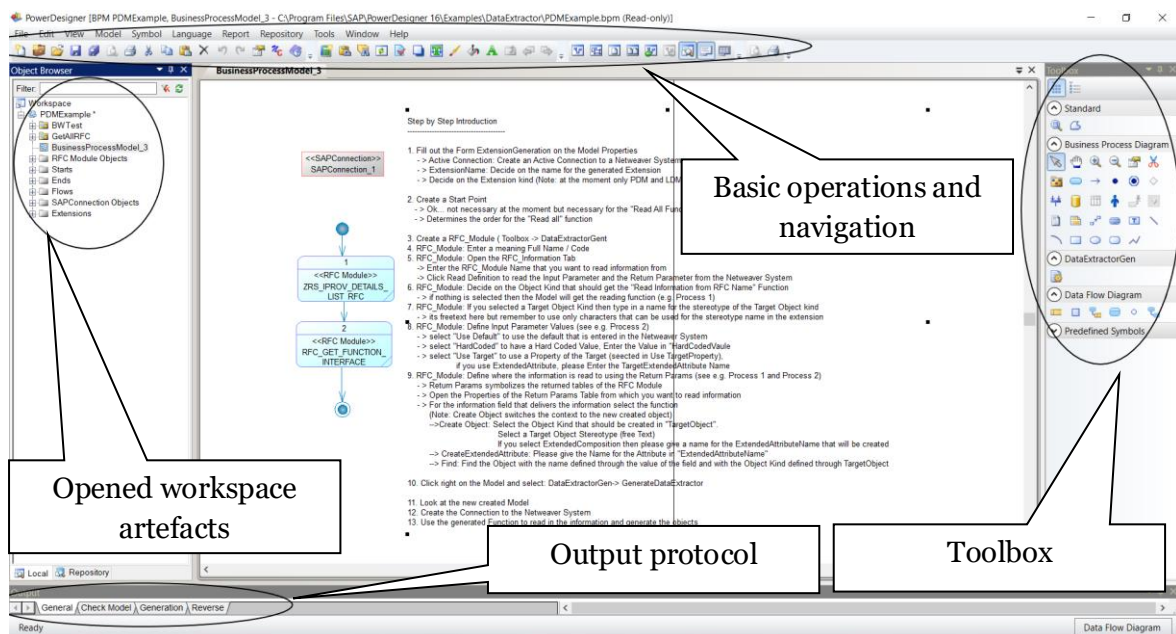


Figure 3: Power designer v16.7 workspace

It allows modelling all kinds of diagrams using standards such as UML or BPMN, to compare and merge them, and of course to transform them into various artefacts for different platforms.

Power designer is often considered as one of the best tools for database design. It is also highly featured for great metadata management or great amount of possible customization.

2.3 Rational Software Architect

Developer: IBM (originally Rational Software)

RSA is Eclipse-based integrated design and development environment which utilizes MDD principles with UML. It is not the first application presented by Rational Software, but currently, it is the most popular. First version was released in December 2006.

2.3.1 Pricing

IBM Rational Software Architect offers 8 different licenses. There is no difference in functionality range, but they are distinguished as User or Floating licenses and they all come with 12 months support. The prices then differ from \$1,290 to \$17,500.

2.3.2 Functionalities

Rational Software Architect features allow to transform created models to Java and C++ source code, or other runtime artefacts such as WSDL, XSD, configuration files and many more.

It supports Business Process Model and Notation (BPMN), UML with addition of UML domain specific notations, but also simple sketching.

Between main advantages of this tool reviewers mention mainly good integration with MS Office products and Eclipse plug-ins support. On the other hand, as a downside is mostly considered the not very intuitive and outdated user interface, or small amount of supported DB platforms.

2.4 Conclusion

All described tools are very popular modelling tools with a great range of functionalities. They are all aimed to build big and complex solutions and they are all very successful at it.

When pricing is considered, Enterprise Architect is unexpectedly the far most cheap option. This might be a big difference, mainly for smaller companies, start-ups or freelance developers.

Nevertheless, they all may have one common feature, which may be a drawback in some cases – they are so complex and have so many functionalities, that it must be very difficult to set and specify boundaries and techniques for any consistent development. And as mentioned in previous chapters, this can be a deal breaker for model-driven software development project.

3 Metasystem application

In this part of the thesis, I would like to introduce Metasystem (further also MTS) tool, briefly describe its' main functionalities and usage, with one main question – would MTS be competitive on commercial market?

The Metasystem is an analytical tool used for the development of multi-functional large information systems. It is a non-commercial application, developed and maintained used in Czech IT company Solitea Business Solutions, s.r.o. (originally Aquasoft s.r.o).

For answering the stated question, I try to model a very simple application in one of the tools I mentioned in the previous chapter and then in the Metasystem application and then analyse the outcome.

3.1 Origins and motives

The main motive for the development of such a tool was the gap between analysts and programmers. As most products of the mentioned company were tailor-made applications, the importance of business analysts was always quite high. First, analysts spent weeks by defining lifecycle diagrams, business process models, functional specifications and all such documentation. After the documentation was agreed with the customer and all involved parties, it was handed over to the programmers – but often, the programmers did not have the same view and the same understanding of the business. This resulted in many iterations of discussions and testing and re-programming the functionality until all parties were satisfied.

The cost of transferring the information and the business know-how from analysts to programmers was simply too costly. It also seemed that there is a lot of work done, but there is so little practical use of it. This led to an idea to create a tool, which will automatically transform the created models to the native code, based on model-driven development principles.

3.2 Framework and functionalities

Metasystem application is part of the custom-developed framework. It is based on Microsoft .NET technologies and provides full support for the development of the final product – databases, backend application (which are modelled in mentioned Metasystem tool), client applications and the whole infrastructure behind it.

The framework also covers the whole process in which the outcome from the MTS tool is transformed to the resulting C# code and deployed to a selected environment.

The framework main principles are:

- Object-oriented approach, the persistence of the objects is handled by the relational database.
- Message-based applications: all actions are triggered by an incoming message, most commonly serialized into XML format.
- State machines: the main “root” objects have a predefined business lifecycle, which is handled by defined automat.
- The behaviour of the applications does not need to be hardcoded by programmers but can be modelled and scripted by analysts, based on model-driven development principles.

Metasystem supports strictly object-oriented development. It offers most of the functionalities needed for application development. Each project handled in MTS consists of separate packages with artefacts like classes, messages, code lists, state automats, rules and conditions etc. These packages can be shared between more projects; this principle allows sharing and reusing definitions.

The basis for automated code generation is a so-called application template, which is maintained by programmers and according which the models are transformed to the resulting code.

Metasystem allows modelling of both static and dynamic behaviour – mainly because of embedded “Action script language”, which is a scripting language used in the modelling tool, later transformed into native code. As it is limited in comparison with standard programming languages, not everything can be scripted in the tool, nonetheless it is possible to implement the needed parts straight to source code in native C#.

In optimal cases, approximately 95% of the resulting application can be modelled in MTS; the rest must be done by a programmer.

3.3 Comparison with commercial tools

For the comparison, I made up a very simple and small application and I modelled it using the Enterprise architect described in chapter 2 and then using the Metasystem application.

The modelled application should serve as a server-side component for some kind of electronic student's book, where the teacher can assign marks and homework for the student. The key promise is that the client part is a web application, and client-server communication is based on web services.

My targets were following:

- model the class diagram
- design the incoming message from the client, which adds new assignment to the student's record
- design the behaviour of the application after the message arrives (save it and send a notification to parent, if needed)

At this point, I abstracted from the solution architecture, platform and used technology; I focused only on modelling the behaviour of this application.

The comparison is done mainly based on the following aspects:

- user-friendliness of the interface
- the range of (useful) functionalities
- flexibility for different projects
- the amount of automation for generating the resulting application

3.3.1 Metasystem

Creation of a new project in MTS is very simple. On the other hand, before modelling of any project can start, there is quite a lot of work for the programmers maintaining the framework – mainly the application template, according which the resulting application is generated (more on this in chapter 3.2).

This is one of both advantages and disadvantages: on one side, it allows great flexibility and customizable features; on the other hand, it takes additional programming to maintain it.

The basics of each project – the common metatypes, common data storage types (persistent and not persistent) and state machine for error processing is in a special package, which should be imported at the beginning.

Class diagram

I began with the definition of a class diagram.

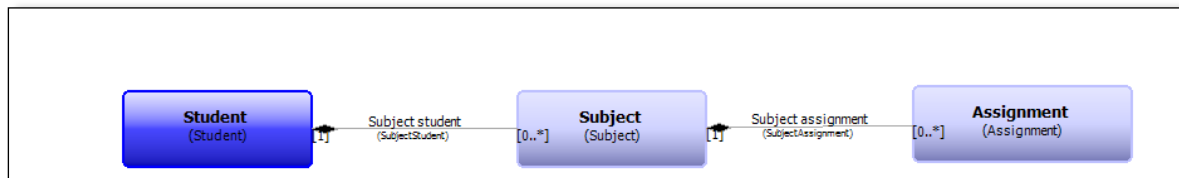


Figure 4: Class diagram in Metasystem application

MTS allows all basic needed functionalities for classes definitions. Each class can have defined attributes, constants and methods. All definitions are public by default.

There is one major restriction, and that is the associations modelling. Currently, it is only possible to define compositions¹. This feature reduces the possibilities of how to model the resulting application, but in our example, this is not an issue.

One great feature is the possibility to very easily model methods of each class by scripting.

¹ Type of association, where one of the classes is always a parent with exactly one repetition, and the child class can exist (zero to multiple times) only, when the parent one exists.

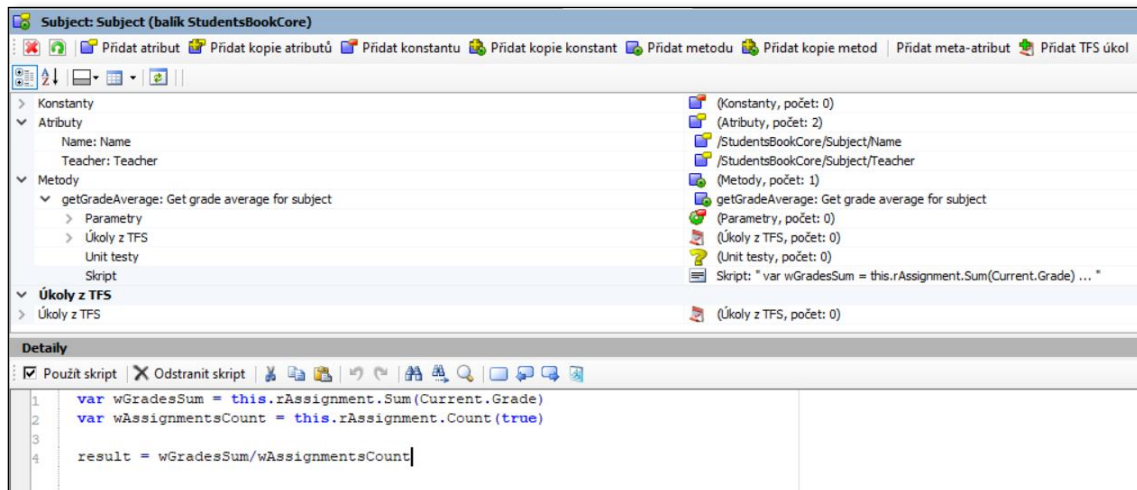


Figure 5: Metasystem class method definition

From modelled class diagram (and selected persistence type), the database model and SQL script are generated during the compilation. There is no need for the modeller to define primary and foreign keys, everything is generated automatically – this also means, that all primary keys are by default surrogate. Additional changes can always be made by a programmer straight to the database scripts, native code or application template.

Client message exchange definition

As the whole framework is message-based, Metasystem offers quite extensive support for message definition, including inheritance, which allows greater code reuse.

The defined messages structure can be then serialized into XML or JSON, again depending on the configuration in the application template.

MTS allows a great way of persistence definition through mapping – it has to be defined for each group and element in the message, but then it can be called from all transitions or methods. This allows an easy way of saving the message data for incoming messages or filling the data for outgoing messages.

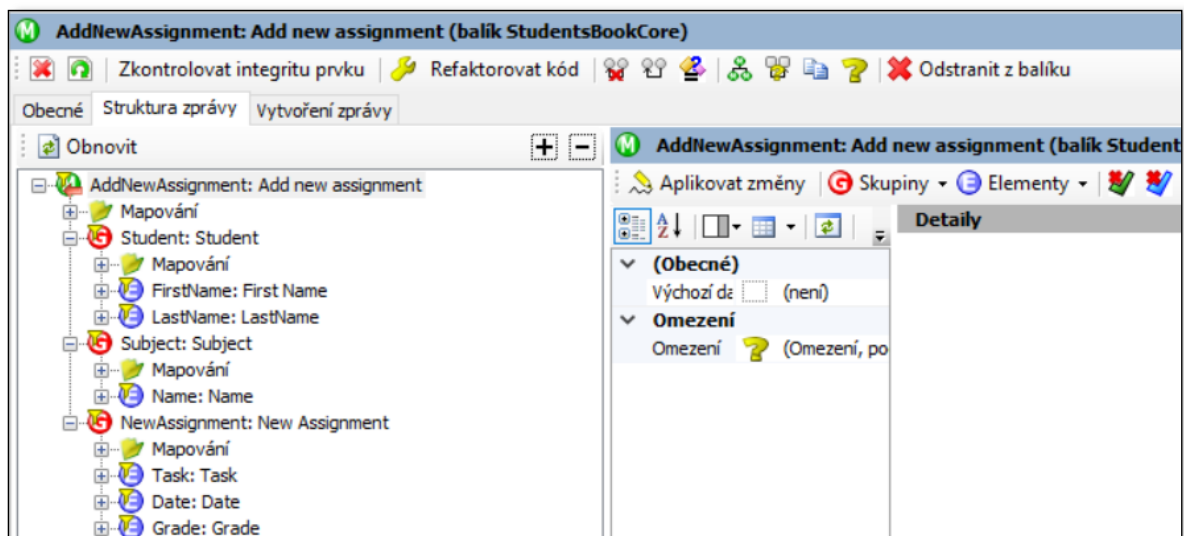


Figure 6: Metasystem message definition

The developer can add all kinds of restrictions for each group or element – metatypes, code lists, enums or scripted rules and conditions.

The XSD of the message or whole WSDL can be generated during compilation. Other types of documentation, like Excel or HTML files, can be generated straight from the modelling tool.

Add new assignment transition

For behaviour definition, Metasystem uses state machines. The common workflow is that after the message is received to the application queue, a specific object instance is connected and for it, state machine and correct transition in it is selected. There can be many machines with many transitions, so it is up to the developer to define by scripting, which will be selected. The main premise is, that each class has some “state” attribute, according which the state machine is driven.

For the chosen example, this creates a slight issue, because there is no business state, according to which the machine could be driven. This is another possible drawback of the Metasystem application, which makes it advantageous only for specific application types, mainly with strictly defined business workflow. Nevertheless, it is not a dead-end situation.

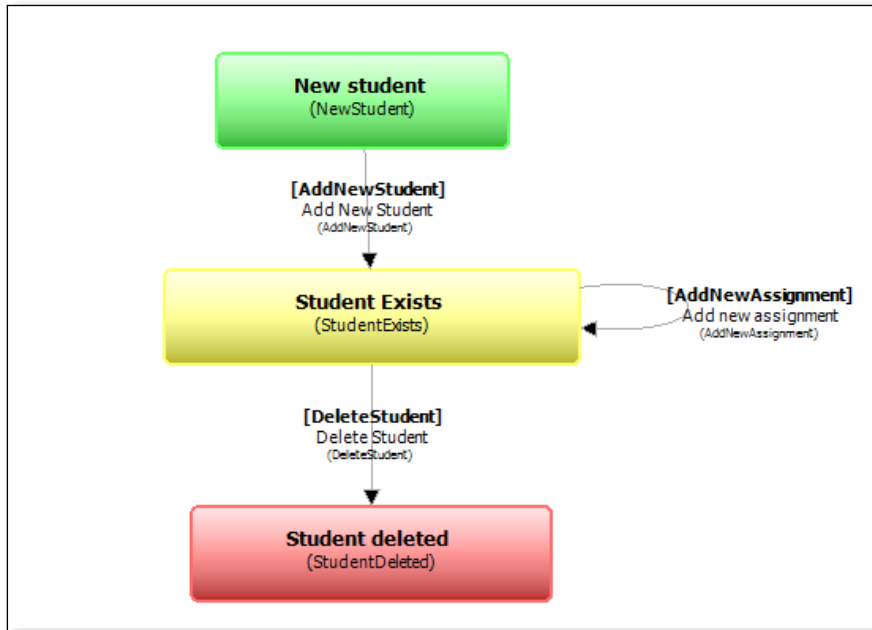


Figure 7: Metasystem state machine

I have created a state machine, which is driven by the Student record existence. Each transition is triggered by a specific message. In this case, message AddNewStudent creates a new record and moves the instance from the initial state (where instance does not exist), to state StudentExists. Here, all of the client communication will take place (as message AddNewAssignment in the example). Once the Student record is no longer needed, message DeleteStudent will mark the instance as deleted.

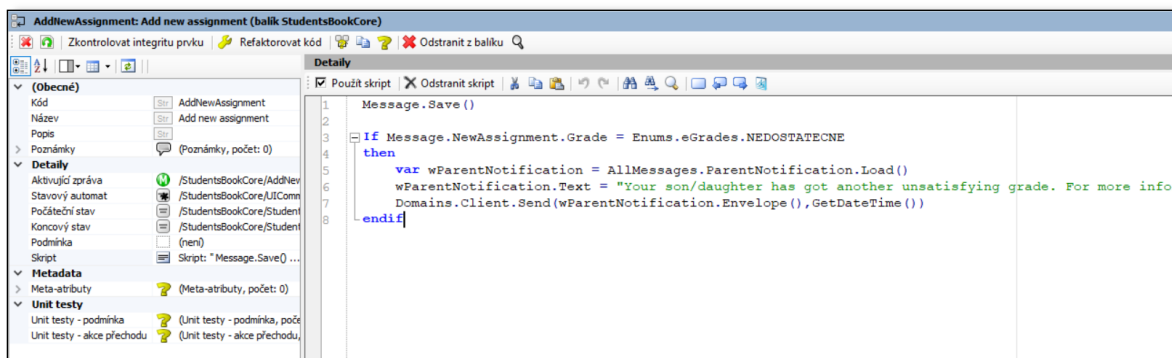


Figure 8: Metasystem transition definition

In each transition, the developer can define by scripting the required behaviour of the selected instance. In our example, I save the data from the incoming message (the saving is defined by the mapping described in the previous chapter) and if the grade is unsatisfying, the application generates and sends a notification to the parent.

This allows the analyst or developer to define the majority of the behaviour of the application, without any deep knowledge of programming. The mentioned combination of static and dynamic behaviour definitions is probably one of the biggest advantages of this application.

Summary

For a brief summary of the Metasystem behaviour, I prepared the following lists of advantages and disadvantages. The application, of course, offers many more functionalities and possibilities, the following evaluation is made mainly based on the features used in this sample scenario.

+	-
• very simple and clear design, easy to use and learn • good support for messages definition, user-friendly visualization, easy persistence definition for message data • customizable by adjusting the application template and other native parts • analysts/developers can define the majority of the behaviour (both static and dynamic) by just modelling and simple scripting – very powerful code generation	• message driven framework – useful mainly for client-server architecture • majorly instance state-driven, useful only for applications with strictly defined business lifecycle • no class inheritance, which limits code reuse • limited options for version control and team collaboration • limited options for class models (only compositions allowed)

3.3.2 Enterprise Architect

For this study, I used a 30-day free trial of EA application.

Class diagram

The modelling of the class diagram using Enterprise architect was quite easy and user friendly. It allows all needed functionalities and modelling techniques, including a range of features for documentation and collaboration purposes, like adding files and notes and links to each defined object.

EA allows defining separately class diagram and database model, which in some cases (as in the example), may require duplicate work. On the other hand, the Database builder mode offers much more possibilities than only class diagram modeller – mainly definition of views, constraints, procedures, functions.

Before a user can begin to design a database diagram, a required platform has to be chosen. In our scenario, I used MySQL DB. The first step is to design a conceptual model of the database. In this case, I basically replicated the class diagram, which created duplicated work for me.

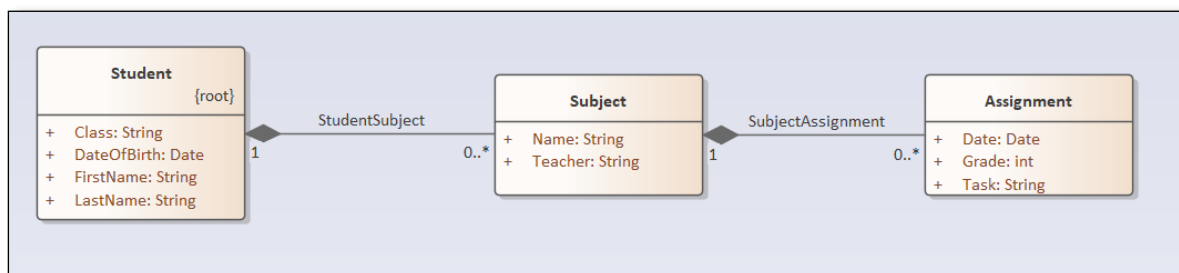


Figure 9: Enterprise Architect Conceptual model

The following task was to design a logical model. What I missed, was an easy reuse of already modelled objects in the package. Therefore even though it describes the same subject as previously created artefacts, only on different levels of abstraction, the model had to be created again from scratch. It is possible to define custom templates for transformations between any objects, but I did not use this option, as it requires quite advanced knowledge.

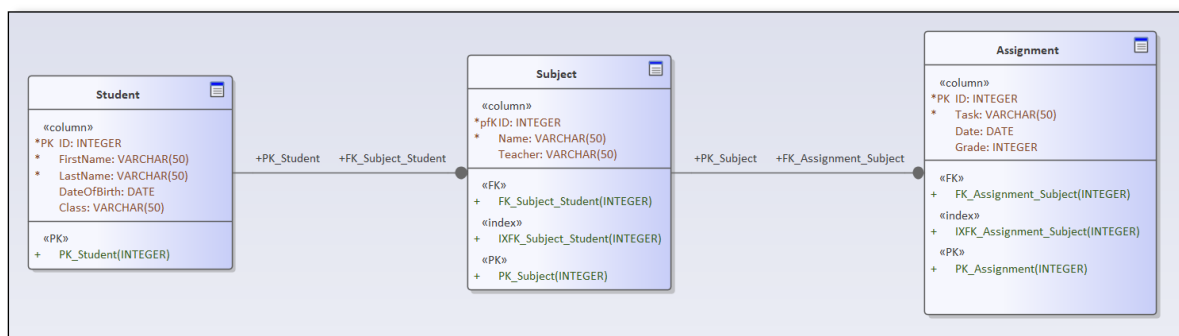


Figure 10: Enterprise Architect Logical model

On the other hand, EA offers very advanced functionalities and possibilities for data modelling, which mainly advanced developers may utilize.

At this point, the user is already able to easily generate the DDL script for the target database platform.

Client message exchange definition

EA allows generating message definition (in XSD format) from the composed data model. Nevertheless, it does not offer user to model the content of the message itself, which limits the possibilities on any interface definition – in EA analyst can model communication diagram, sequence diagrams and many more, but the contents of the messages will have to be defined separately. Even if we import and already defined WSDL, it is not possible to see the contents of the messages.

Add new assignment transition

For the definition of the application behaviour, I used sequence diagram and communication diagram. In the first diagram, I defined a general communication workflow between the components and in the second, a detailed procedure between the object instances.

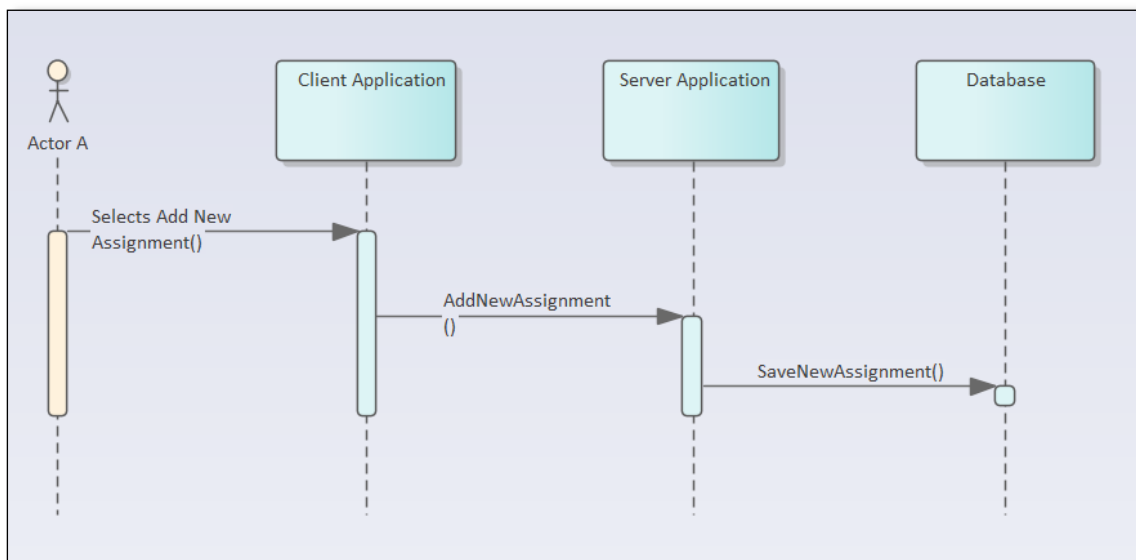


Figure 11: Enterprise Architect Sequence diagram

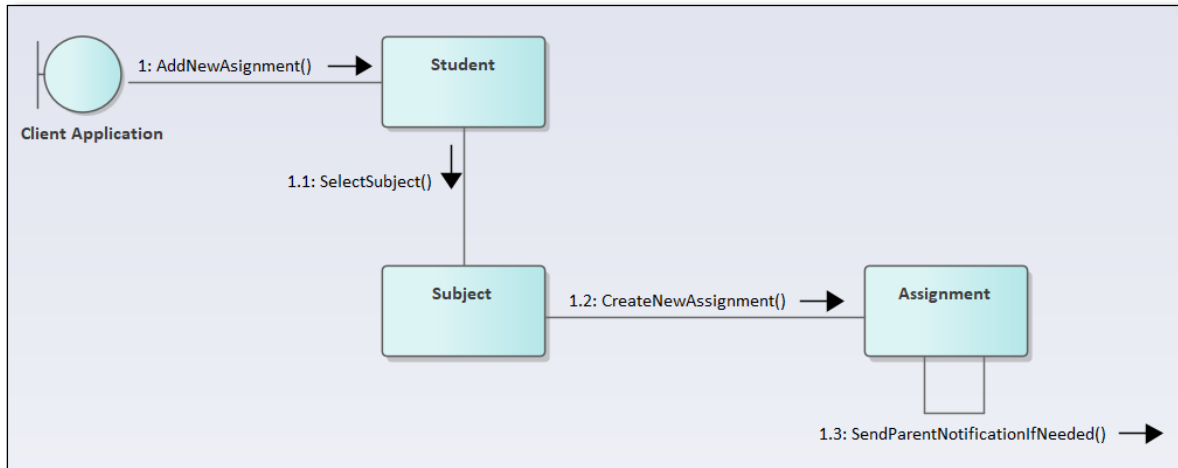


Figure 12: Enterprise Architect Communication diagram

If the modeller correctly connects all diagrams and instances in the project, Enterprise Architect very easily generates the code which then can be used in the resulting application.

Summary

Enterprise Architect is, without any doubt, a very powerful tool. It offers a number of functionalities, highly advanced modelling tools and a wide range of features simplifying team collaboration and project management in general. Nevertheless, for a person who never used it for his or her own development, it was sometimes quite hard to find the needed functionality – even with documentation provided on the internet.

Again, the following evaluation is based mainly on the functionalities I have come across and used in this example scenario.

+	-
• wide range of functionalities and advanced modelling tools • highly customizable, can generate resulting code for many platforms and programming languages • features for team collaboration, version control and project management	• sometimes confusing and complicated user interface • limited options for messages definition • high pricing, which makes it not suitable for non-commercial smaller projects

3.3.3 Comparison and summary

After performing the previous testing scenario, I have come to one conclusion: it is really hard to compare the mentioned two tools because even though they both aim for the same thing, they are quite distinct.

Metasystem can be very powerful mainly because of a very high level of abstraction, which allows that even analysts without any deep programming knowledge are able to model the system behaviour and the automatic code generator performs most of the work. It has certain limitations, so it is not advantageous for every project, but if it is used for some specific type of applications which it was designed for, it is a very robust tool.

It also offers a good combination of modelling static and dynamic behaviour – mainly because of build-in Action Script support, which is then transformed straight to the native code.

One of the main advantage which should be emphasised is that in comparison with other commercial modelling tools, it enforces unification and consolidation of the used procedures and modelling techniques. As it has quite strictly defined boundaries which the analysts must obey, it minimizes the error rate and simplifies the code generation process. This ensures, that the automated transformation can produce majority of the resulting application.

Mentioned fact could be of course also taken as a drawback. However, the main assumption in this case is, that this is a tool mainly for analysts, who usually do not have an extensive programming knowledge. If the developer is skilled enough, it is always possible to write a native code, which is then integrated to the output from MTS.

On the other hand, EA offers much more advanced modelling tools and techniques, which make it suitable for basically any project, no matter whether solely for documentation purposes, or also for the development itself. However, it still requires a lot of additional work to customize it and to link the created models with the application behaviour.

In the current situation, I do not think that Metasystem would be commercially competitive. It is so narrow profiled and specific, that it would have to be implemented with the whole framework by the specialists who are developing and maintaining it, because without it, it is basically useless. Moreover, it would be probably quite hard to break through with the current market situation, where not only Enterprise Architect is very successful.

Although, it does not mean, that the tool is not powerful and useful enough. For certain types of applications - large server-based applications providing extensive communication with surrounding systems, it is much more profitable and convenient, than other mentioned applications. Nonetheless, the price for “making it commercial” and maintaining it outside the origin company would be probably still too high at the moment.

4 Analysis and design of new MTS functionality

In the last part of the thesis, I analyse requirements and design a new functionality for Metasystem application, which could possibly eliminate some known shortcomings and enhance the usage of it and contribute to better possibilities for the analyst or developer.

As complex as the tool is, there are several areas, which could be improved. It was discussed this with other analysts and developers working with Metasystem and here are the most common outputs:

- Better inheritance and polymorphism support: allow inheritance of classes and abstract definitions. Currently, MTS offers this only for messages. This could lead mainly to better code reuse.
- Extended features for team collaboration and version control: for example, graphical distinction for common packages used between several projects or automatic project data refresh after Git pull. Main aim would be to reduce amount of issues caused by incorrect code handling.
- To allow modelling of all types of associations (currently, only compositions are supported): this would extend range of possible target applications, nevertheless it would also require great changes in behaviour of many other artefacts as state machines and message mappings.
- Enhancement of message modelling functionality: the goal would be to allow the developer to apply certain actions (such as rules and conditions removal) in batch to multiple nodes. This would save the additional work which is needed to manually do the needed action on each element.

After some evaluation, I choose for further analysis two options: the first, inheritance and polymorphism, and the last, message modelling enhancement. Class inheritance mainly because it is one of the main principles of object-oriented development in general, and the second as it could ease some tasks which currently require an unnecessary amount of time. Both are features, which could significantly help already ongoing projects.

The second variant (team collaboration features) is not that essential and probably would require only small changes. The third one (extended associations modelling) would be without any doubt quite interesting and would make the application more competitive on

the market, however for the projects it is used for at the moment, this is not a necessary feature and what is more, it would probably require redesign and extension not only of MTS tool, but also of the whole code transformation process and framework.

4.1 Class inheritance and polymorphism

Inheritance and polymorphism are one of the basic principles of object-oriented programming. It is supported by majority of the significant OOP languages, such as Java, C#, C++, Python or PHP. Inheritance is used mainly on classes and attempts to maximize code reuse by sharing and inheriting properties between objects, while polymorphism allows to store certain artefact, such as method, in different forms and when called, use the one variant which is suitable for the specific object that initiated this action.

4.1.1 Initial study

This project aims to analyse and design new features into the Metasystem application. The newly implemented functionality will allow using class inheritance and polymorphism in the framework.

Main benefits

- better code reuse
- reduction of duplicate work for the modeller
- greater conjunction and coherence between different applications of the same solution (for example in case of more back-end modules in one project)

Main objectives

- enable generalization relationship between classes and implement inheritance functionality into the Metasystem modelling tool
- add class inheritance support to the framework and adjust code generation accordingly
- ensure backwards usability so that this feature can be used even for already implemented parts of the applications
- train the analysts working with the tool for the best utilization of this feature

Expected schedule and workload

As the project shall be quite small and maximally two developers are expected to participate, the work is expected to be organized as in standard waterfall model. The workload of specific phases is calculated in MD².

Phase	Workload
Requirements specification	1 MD
Analysis and Design	2 MD
Implementation	17 MD
Verification	5 MD

Which in summary creates work estimation for 25 MD, therefore the whole project should be finished after about 5 weeks.

4.1.2 Requirements specification

This chapter shall cover the requirements specification. It is divided into two parts – functional and non-functional. Each requirement is described and evaluated on a scale from 1 (least needed) to 5 (most needed) by its importance for the final solution.

Functional requirements

Nr.	Functional Requirement	Priority
1	The system shall allow class inheritance	5
The Metasystem application shall allow creating a class, which is a subclass to another class. Multiple inheritance is not allowed. The subclass will acquire all properties (attributes, constants and methods) of the parent class (superclass). Inheritance and all its features shall respect C# principles.		
2	The system shall allow defining abstract classes	4

² Man-day: one person's working time for one day, standard unit of measure for workload of any task.

The Metasystem application shall allow marking a class as abstract. This class will not be instantiated. An abstract class can be both subclass and superclass in the inheritance context. Abstract classes cannot have any persistence method assigned so they cannot be added to class diagrams.

3	The system shall allow modelling a method as abstract	5
---	--	---

It must be possible to mark a method as abstract. This method will not have any implementation and will serve only as a template for derived classes. Derived non-abstract classes must implement this method by overriding it. All methods are by default marked as virtual; there is no need to explicitly state it.

4	The system shall allow and enforce method overriding	4
---	---	---

It must be possible for inherited methods to override them in the subclass. In cases where derived class inherits abstract method and is not marked as abstract itself, the overriding is enforced by the system.

5	The system shall allow method overloading	2
---	--	---

The Metasystem application shall allow to overload inherited classes. Overloaded method must differ from its parent by number, type or order of parameters.

6	The system shall allow marking any class as a subclass backwards	3
---	---	---

In order to allow backwards compatibility with already developed parts of the application, Metasystem shall enable to mark any already existing class as a subclass to different class. All methods and properties will be kept; the inherited will be added.

7	The framework shall support the class inheritance and corresponding code generation	5
---	--	---

It will be possible to generate code for resulting application in the way how it was modelled in Metasystem application.

Non-functional requirements

Nr.	Non-functional Requirement	Priority
1	The system shall allow generating documentation in accordance with the new functionality	5
The system will be able to generate documentation, which is in accordance with the newly implemented functionality.		
2	The system shall notify the user about any errors in the models	4
The Metasystem application shall correctly notify the user about any errors in the models. New integrity checks regarding the class inheritance will be added.		
3	The system will graphically distinguish abstract classes and methods	2
The metasystem application shall graphically distinguish abstract classes and methods, perhaps with the colour of the class/method icon.		

4.1.3 Analysis and design

This part of the study covers the analysis and design of new functionality.

The main aim of the newly added functionality is to allow the developers to model class inheritance.

General functionality

The system will allow creating a derived class by adding an option “Add new subclass” into dialogue opened by right-clicking on the “Classes” artefact in the package.

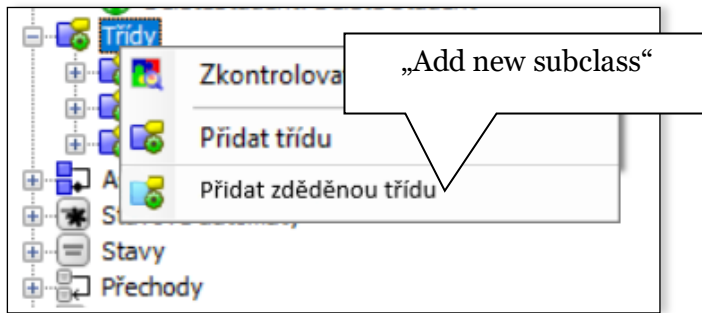


Figure 13: Metasystem "Add new derived class" dialogue

After selecting the “Add new derived class” option, the user will be prompted with a window to select the parent class.

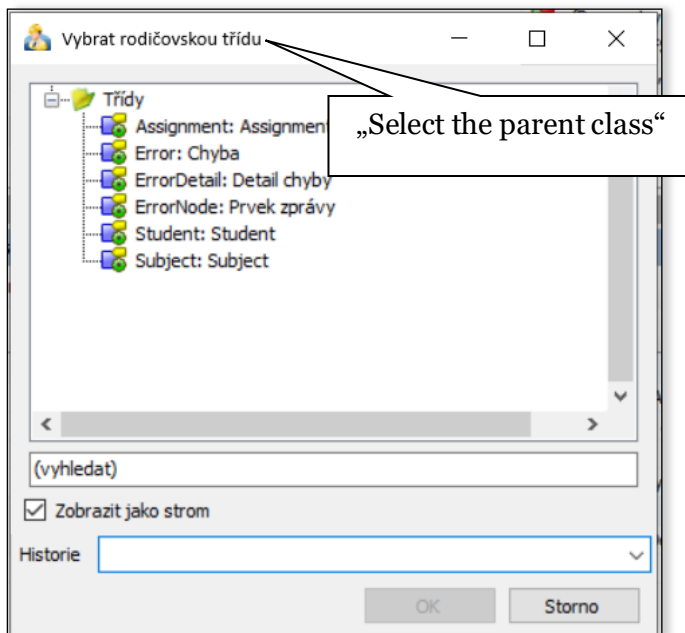


Figure 14: Metasystem "Select the parent class"

After selection of the parental class, a new definition is created and a detail of the artefact opens.

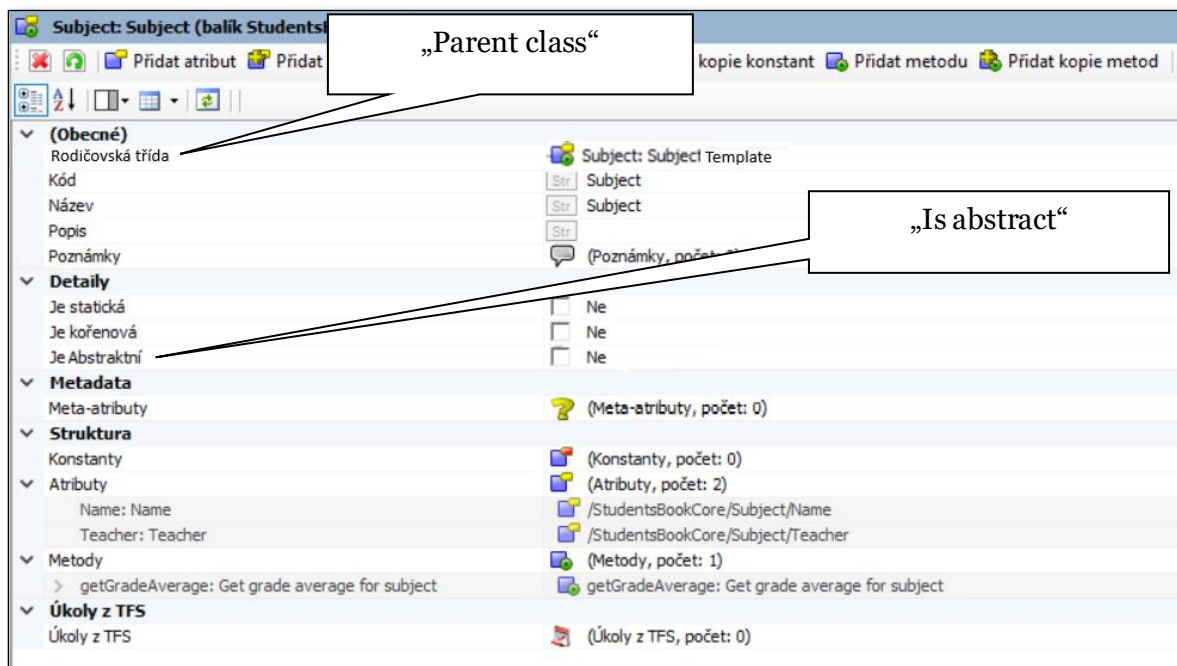


Figure 15: Metasystem Class artefact detail

The inherited attributes, methods and constants shall be by default disabled and read only.

Methods overriding

If the developer opens the detail of the inherited method, there will be checkbox „Override“, which will enable the script and allow to write different method implementation. If user unselects “override” checkbox, the method script will be disabled and returned back to the original state.

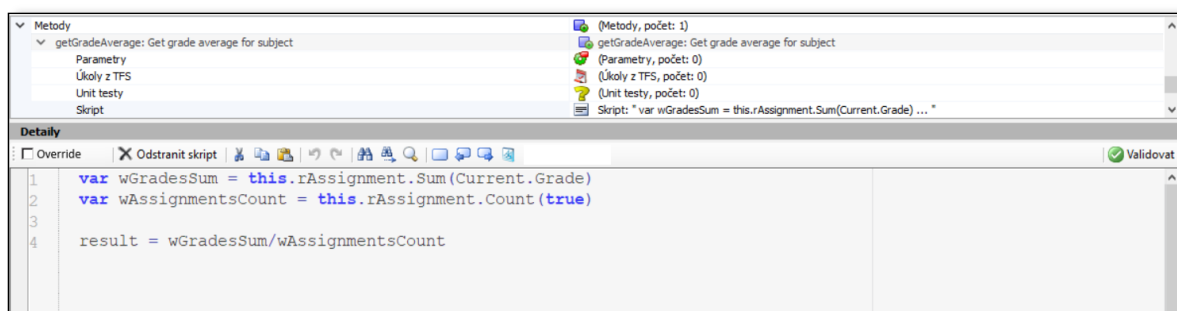


Figure 16: Metasystem Inherited method detail

If a method needs to be overridden, because its ancestor was abstract method, “Override” checkbox will be selected by default and disabled, so that user cannot uncheck it.

For methods which are not inherited, override option shall be disabled.

Methods overloading

“Add overloaded method” option shall be added to class artefact detail, next to other action buttons.



Figure 17: Metasystem Add overloaded method

After selecting this option, the system shall offer dialog window with the possibility to select the method for overloading. It needs to be method from the class context.

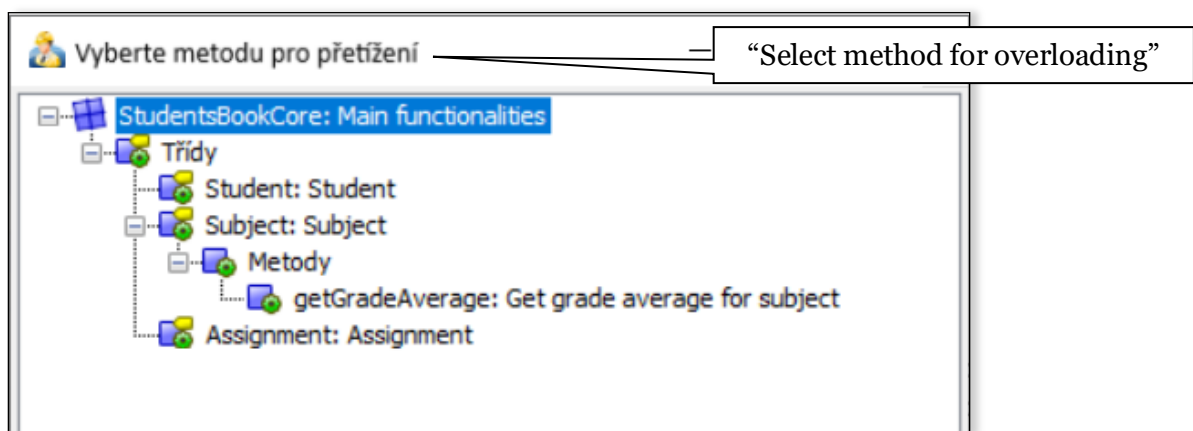


Figure 18: Metasystem Select method for overloading

While scripting, the help should offer all possibilities of the method, switching between them using Ctrl+Shift+Space (as it is currently for native overloaded methods).

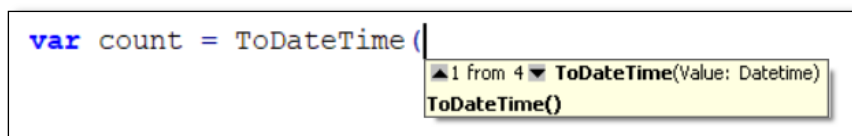


Figure 19: Metasystem overloaded methods selection

4.2 Message modelling enhancement

Message modelling is done by actions triggered by the user. All the user selects to do, is recorded and transformed into so called builders.

Builders are an artefact, which is framework specific and is not commonly used in programming. They serialize the procedure of creation a new message. In most cases, they are automatically generated based on the user actions, nonetheless to some extent, they can be also manually updated. They ensure, that the message instance will be always generated in up to date form, based on the state of associated artefacts, such as ancestor messages in case of cloning.

In the application, they can be seen under the tab “Creation of message”, as it is shown in the following figure:

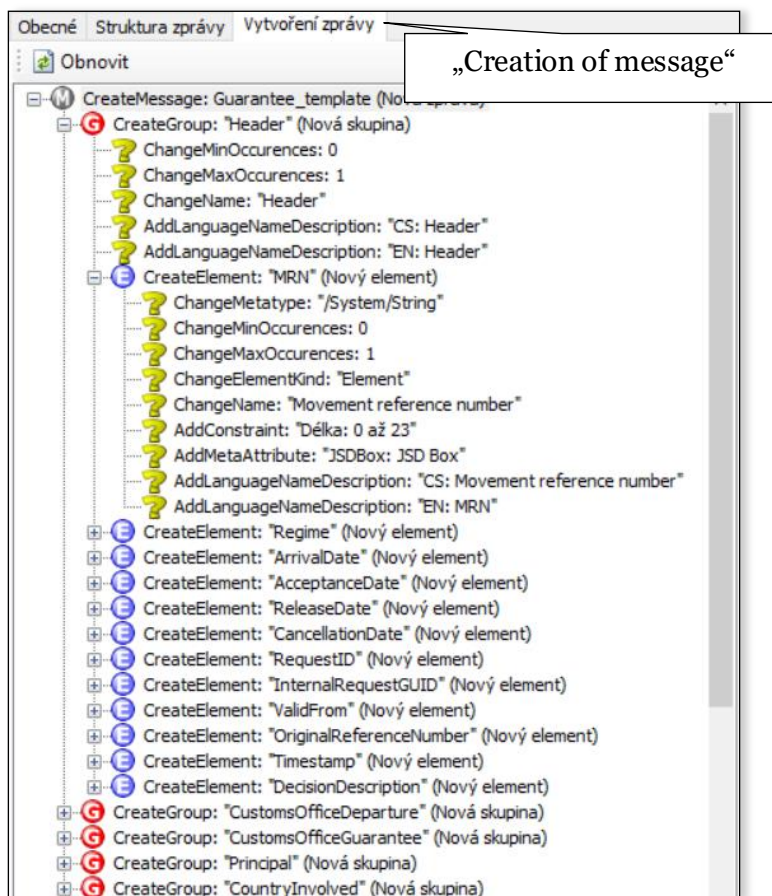


Figure 20: Metasystem Message builders

As it can be seen, each builder has one task: to create new element, to change its name, to add some restriction, to change number of occurrences, etc. Builders are also used for creation or amendment of mapping artefacts, for rules and conditions management, or in general for any action which is performed on the selected artefact.

In case of message inheritance, it is also the builders that carry the information about the ancestor and the cloning manner.

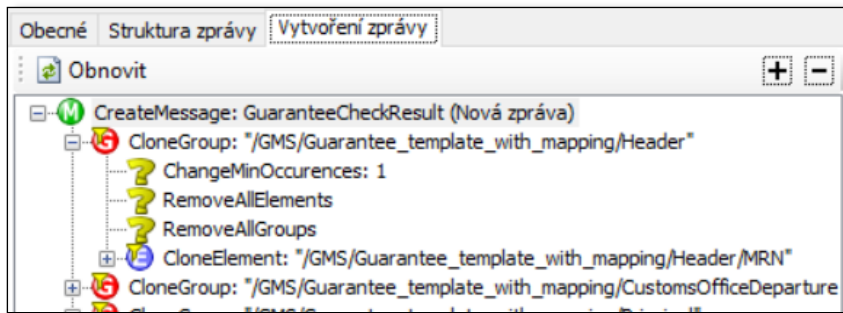


Figure 21: Metasystem cloning builders

Nonetheless, currently, it is only possible to perform needed action on one selected node. Therefore in case of needed batch actions, such as removal of all applied rules and conditions or mapping artefacts, it needs to be manually selected on each element in the message. This can be quite time-consuming task, mainly for extensive messages, such as in the example above.

The possible workaround for this functionality is to manually update the XML serialized metamodel, nevertheless this may easily create unwanted errors and if the developer does not have sufficient experience, it can be even more time-consuming task.

4.2.1 Initial study

Following project will attempt to analyse and design new functionality, which would allow to trigger actions that will automatically use the needed builder for a range of selected elements.

Main benefits

- simplification for the developers
- time-saving feature

Main objectives

- minimization of manually created errors and discrepancies
- to simplify the modelling process for developers
- easier manipulation with extensive messages

Expected schedule and workload

Same as for the previous functionality, probably only one developer will be working on this functionality and in regards to the size of the project, the work is expected to be done according to standard waterfall model.

Phase	Workload
Requirements specification	0,5 MD
Analysis and Design	0,5 MD
Implementation	2 MD
Verification	1 MD

As the functionality is rather small and should not be extensively complicated to implement, we expect approximately 4 days for the whole process.

4.2.2 Requirements specification

Same as for the previous project, in this chapter I will summarize and define the requirements, both functional and non-functional. Each requirement is described and evaluated on a scale from 1 (least needed) to 5 (most needed) by its importance for the final solution.

Nr.	Functional Requirement	Priority
1	The system shall allow to apply an action to whole selected group	5
The application will allow to apply defined batch actions to the whole group with all its child elements. If the group contains another group, it will be applied to it as well.		
2	It shall be possible to manually select elements in the message, to which the action will be applied	3
There shall be an option to manually select all elements, to which a certain action should apply.		
3	The system will allow to apply batch removal of rules and conditions	5

It will be possible to remove all rules or conditions on selected elements (as mentioned in requirements 1 and 2) by applying one batch action

4	The system shall allow to apply a batch removal of mapping artefacts	4
---	---	---

It will be possible to remove all mapping artefacts (both contexts and mapping items) on selected elements (as mentioned in requirements 1 and 2) by applying one batch action

5	It will be possible to apply batch action on the whole message	3
---	---	---

The system will allow to apply a batch action to the whole message artefact. The action will then be performed on all elements and groups in the message.

6	The system shall allow to delete all selected elements as a batch action	3
---	---	---

The system will allow to remove all selected elements (as mentioned in requirement 1 and 2) in one batch action

7	The system shall allow to add specified meta-attribute to all selected elements	2
---	--	---

The application will allow to add one meta-attribute to all selected elements (as mentioned in requirement 1 and 2) in one batch action

Nr.	Non-Functional Requirement	Priority
1	The system shall notify user about any error in the implementation caused by the new functionality	5
If an error appears because of the newly implemented functions, the system shall notify the user in the same way as for any other compilation errors.		
2	The new batch actions shall be selectable in the dialog action window	5
The newly implemented actions will be selectable from the dialog action window, as it is for the rest of user actions.		
3	The batch action will be multiplied as a single builder to each element, so that user can manually amend each one	2

If a batch action is applied to range of elements, it will be implemented as single builders to each of the selected elements. User should be able to manually change a builder, which was created based on batch action

4.2.3 Analysis and design

This chapter will describe and specify the overall behaviour and appearance of the new functionality.

General functionality

During message creation or amendment, both detail of the artefact and the dialog window with actions opened with the right-click shall offer the newly implemented actions.

On a group

As group contains child elements/group and therefore can be a source for any batch action, the dialog window shall provide following:

- Remove all rules on this and all child elements
- Remove all conditions on this and all child elements
- Remove all mapping artefacts on this and all child elements
- Remove all message mapping artefacts on this and all child elements

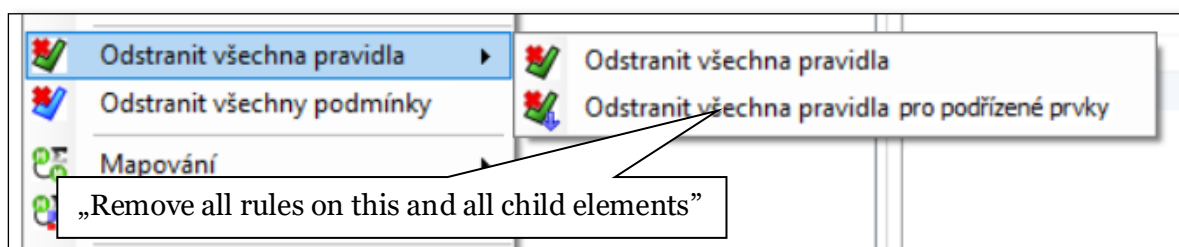


Figure 22: Metasystem "Remove all rules for sub-elements"

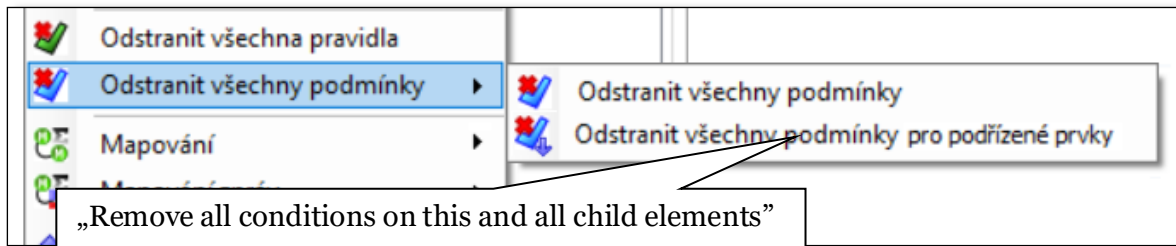


Figure 23: Metasystem "Remove all conditions for sub-elements"

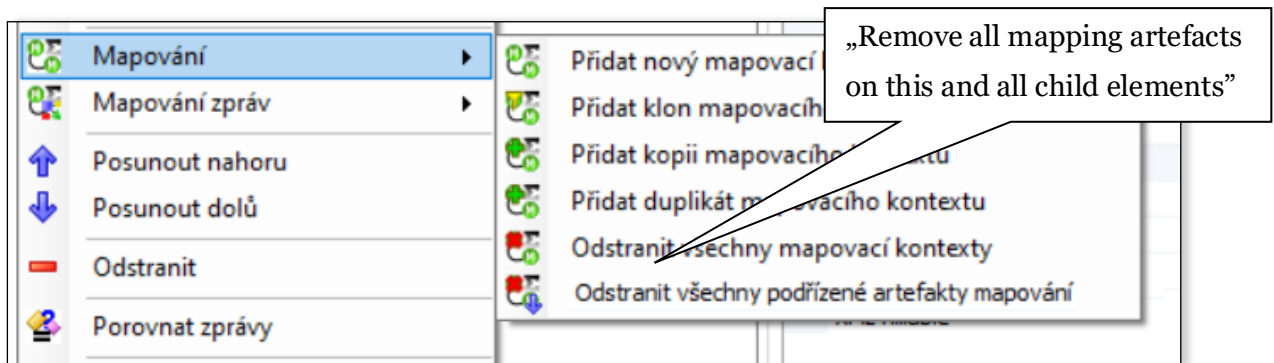


Figure 24: Metasystem "Remove all artefacts for mapping for all sub-elements"

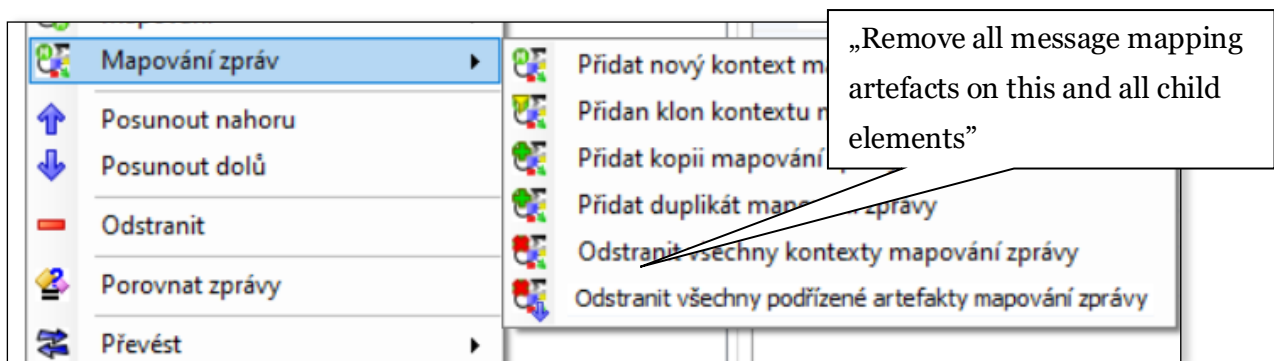


Figure 25: Metasystem "Remove all artefacts for mapping to message for all sub-elements"

The same options shall be offered in the detail window tool-box, as it is for current implementation.

Manually select required elements or groups

It should be possible manually select the required elements, to which the batch action shall be applied. This option could be offered in the dialog window, which is opened after clicking on the root message artefact. The options will be similar as in the cases above, however there will be another option labelled such as "Remove all rules for selected elements".

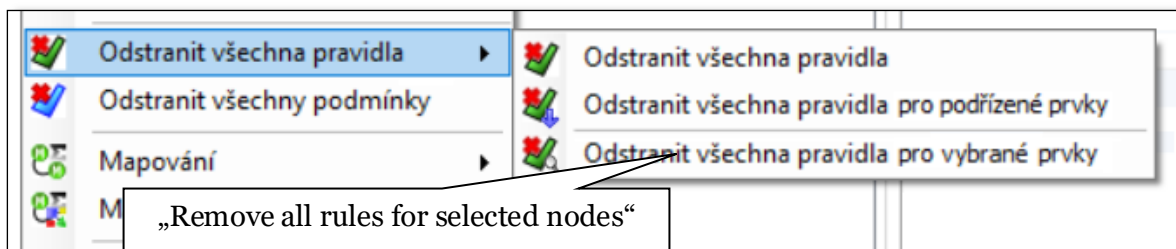


Figure 26: Metasystem "Remove all rules for selected elements"

Selecting this option shall trigger a window with a possibility to select one to many nodes from the message.

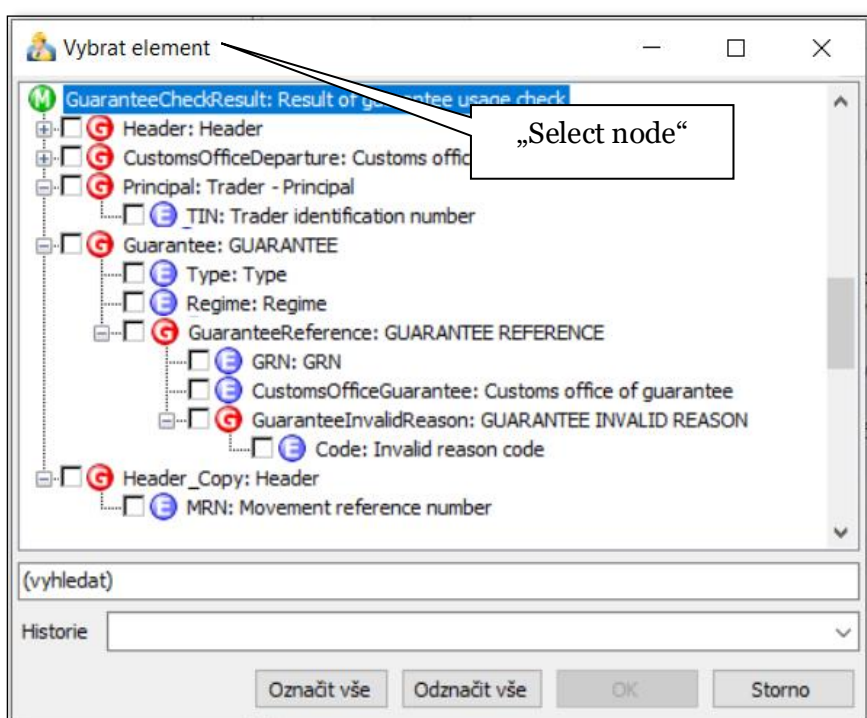


Figure 27: Metasystem "Select element for action"

Selection of a group does not mean, that the action will be applied to all its sub elements, it will be used only for the group node.

After selecting the desired elements, action is applied to all of them.

Builders multiplication

For any batch action which is performed on multiple elements (either on a group sub-elements, or on manually selected nodes), it is desired that the correct builders are multiplied to each affected element, instead of one batch builder on the root element.

This way, the developer can later manually remove the builder from required element, in case the action is no longer desired for it.

If the developer would select „Remove all rules on this and all its child elements, this would be the result:

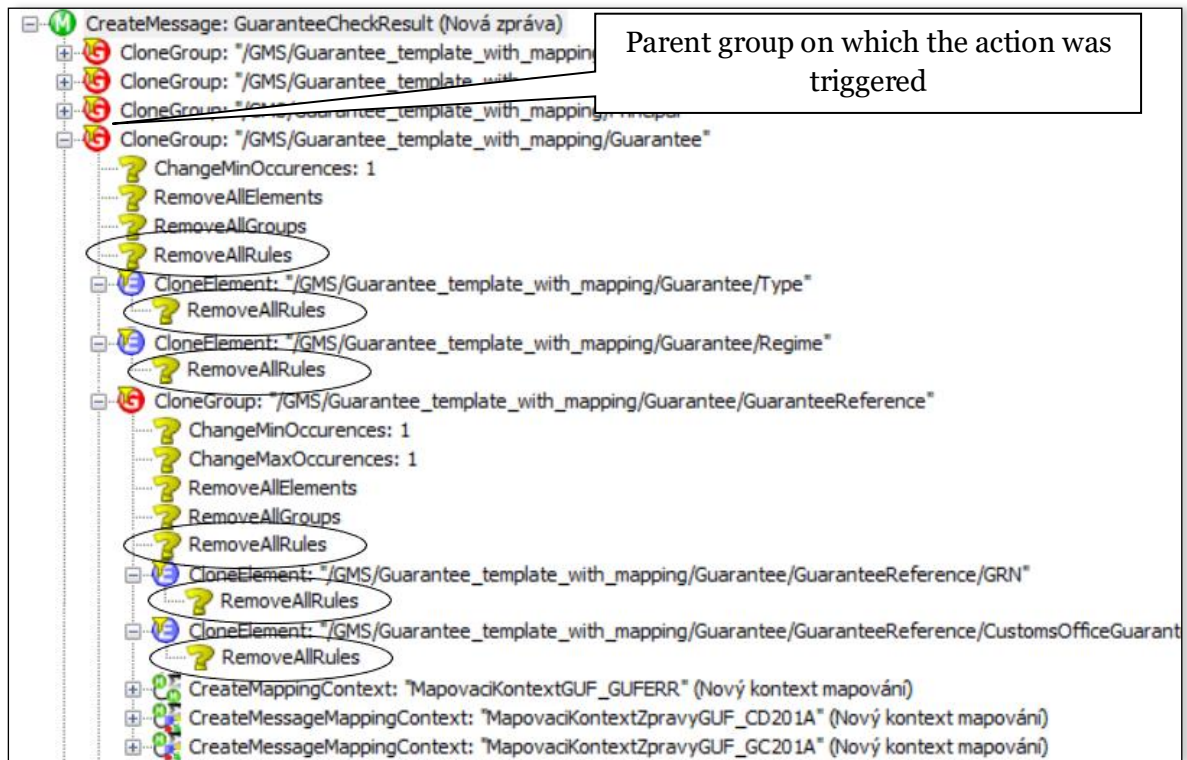


Figure 28: Metasystem multiplication of builders

Conclusions

Development based on model-driven principles and techniques has not been outdated concept and without any doubt offers a range of great possibilities. It is true, that these days, between the trends such as artificial intelligence, cloud computing and more, MDD might not seem that interesting. However for big and complex business SW solutions, it still can teach us great number of things.

As any other software development method, it has both its strongpoints and its downsides. Nonetheless, if it is used correctly and on appropriate projects, it allows creating very complex and business-favourable software solutions.

I see the threats mainly in two following matters: the required, but not easily achievable model precision and the figurative “programmer’s approach”.

In order for model-driven development to bring the desired results, the models need to be precise. Otherwise it is practically impossible to design the needed transformations, which will not make the resulting application code unusable. This requires strictly defined boundaries and used techniques, ideally implemented straight in the modelling tool. However, the bounds cannot be solid and unchangeable – as for anything in software development area, it needs to be flexible, configurable and extendable. As a result, this should support the usage and further evolution of commonly accepted modelling standards and notations.

The other matter I mentioned may not be a technical threat, but the human element in software development process is just as crucial. An experienced programmer will not necessarily be a good developer in terms of model-driven development and vice versa. Despite both activities thrive to achieve the same thing, they obey different approaches.

But despite the mentioned points, if it is approached correctly, the benefits it provides can easily outweigh the threats. The humankind will probably never invent a software development approach, which will suit all projects, all solutions and all teams without any difference, as there are too many variables with too many possible values. Therefore, I hope, that my thesis has shed some light on the arguments, why Model-driven development, despite its drawbacks, is something worth further exploring and investing in.

Terminological glossary

Term	Abbreviation	Meaning (source)
Software development methodology		A software development methodology is a way of managing a software development project. This typically address issues like selecting features for inclusion in the current version, when software will be released, who works on what, and what testing is done. (<i>Software Development Methodologies</i>. Available from:https://www.researchgate.net/publication/255710396_Software_Development_Methodologies)
Information technology	IT	the use of computers to store, retrieve, transmit, and manipulate data or information. (Daintith, John, ed. (2009), "IT", <i>A Dictionary of Physics</i>, Oxford University Press)
Object-oriented programming	OOP	Software programming model constructed around objects. This model compartmentalizes data into objects (data fields) and describes object contents and behavior through the declaration of classes (methods). (Techopedia.com, available at https://www.techopedia.com/definition/3235/object-oriented-programming-oop)
State machine		A state machine is any device storing the status of something at a given time, usually represented by a diagram. The status changes based on inputs, providing the resulting output for the implemented changes. (Techopedia.com, available at https://www.techopedia.com/definition/16447/state-machine)
Artificial intelligence	AI	Artificial intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. The term may also be applied to any

		machine that exhibits traits associated with a human mind such as learning and problem-solving. (Jake Frankenfield, Investopedia.com. Available at https://www.investopedia.com/terms/a/artificial-intelligence-ai.asp)
Cloud computing	CC	Cloud computing is the delivery of different services through the Internet. These resources include tools and applications like data storage, servers, databases, networking, and software. (Jake Frankenfield, Investopedia.com. Available at https://www.investopedia.com/terms/c/cloud-computing.asp)

List of references

1. B. Selic, 'The pragmatics of model-driven development', *IEEE Software*, vol. 2003, no. 5, pp. 19–25.
2. S. Mellor, A. Clark, and T. Futagami, 'Model-driven development', *IEEE Software*, vol. 2003, no. 5, pp. 14–18.
3. B. Hailpern and P. Tarr, 'Model driven development: the good, the bad and the ugly', *IBM Systems Journal*, vol. 2006, no. 3.
4. T. Bruckner, J. Voříšek, A. Buchalceková, I. Stanovská, D. Chlapek, and V. Řepa, *Tvorba informačních systémů: Principy, metodiky, architektury*, vol. 2012. GRADA Publishing.
5. IBM Redbooks, *Patterns: Model-Driven Development Using IBM Rational Software Architect*, vol. 2005. I B M.
6. V. Kulkarni, 'Model Driven Development of Business Applications – a Practitioner's Perspective', *IEEE International Conference on Software Engineering Companion*, vol. 2016.
7. IBM, 'IBM Knowledge Center: Rational Software Architect 9.0.7'. https://www.ibm.com/support/knowledgecenter/cs/SS8PJ7_9.7.0/com.ibm.rsa_base.nav.doc/topics/c_node_overview.html.
8. 'Power Designer Documentation: Core features'. SAP, [Online]. Available: https://www.powerdesigner.biz/documentations/powerdesigner-16.6-documentation-en/core_features.pdf.
9. Gartner, 'Gartner reviews'. <https://www.gartner.com/reviews/>.
10. L. Gála, Z. Šedivá, and J. Pour, *Podniková informatika, 2., přepracované a aktualizované vydání*, vol. 2009. GRADA Publishing.
11. P. Parvaiainen, J. Takalo, S. Teppola, and M. Tihinen, 'Model-Driven Development Processes and practices'. VTT Technical Research Centre of Finland, 2009.
12. Sparx Systems, 'Enterprise Architect, Sparx systems'. <https://sparxsystems.com/products/ea/>.
13. OMG, 'The MetaObject Facility Specification™'. <https://www.omg.org/mof/>.
14. OMG, 'OMG® Unified Modeling Language® version 2.5.1'. Dec. 2017, [Online]. Available: <https://www.omg.org/spec/UML/>.